



# Airflow



# Dagster



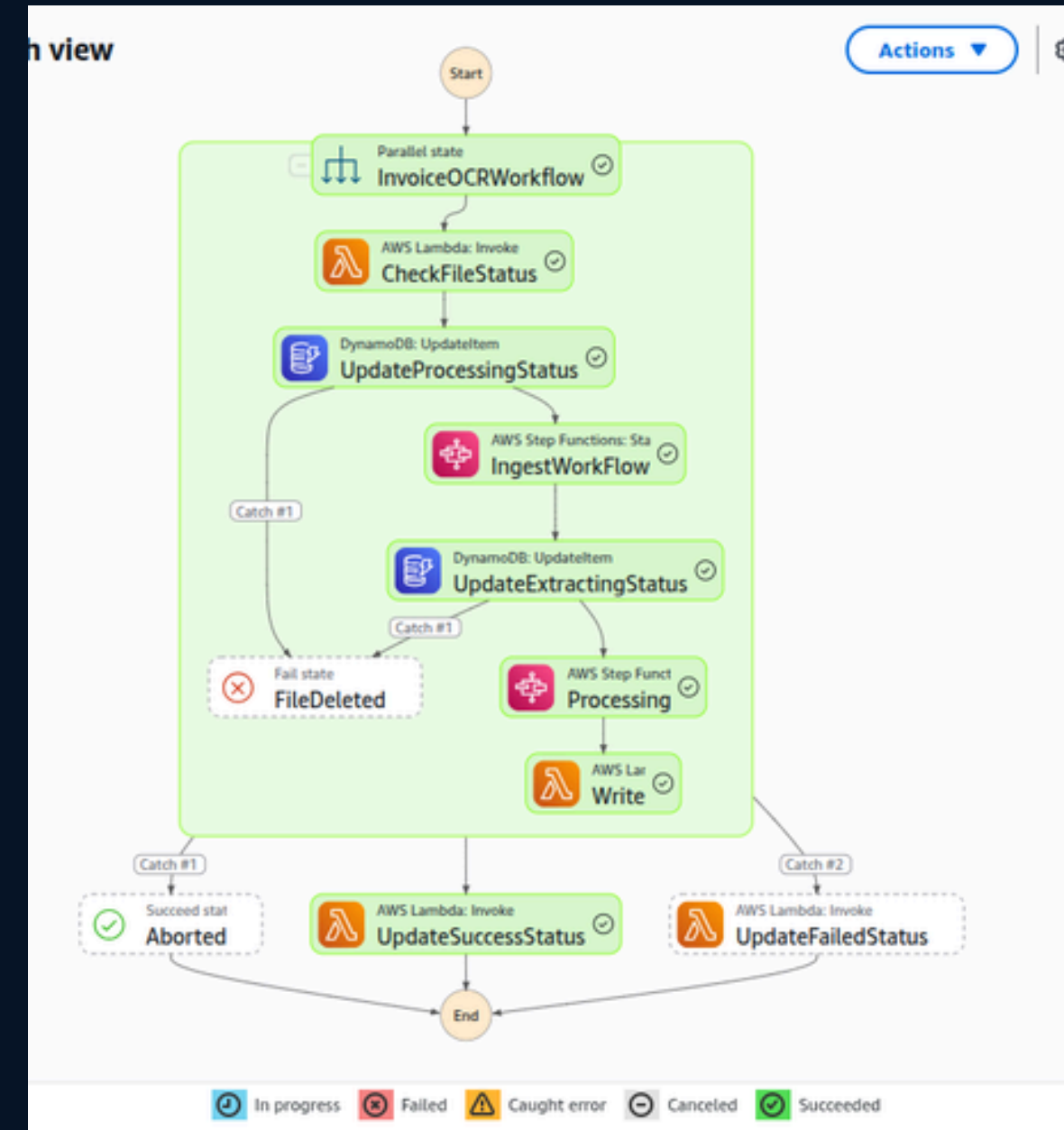
## AWS STEP FUNCTION

### ✓ LỢI THẾ (PROS)

- **Hoàn toàn Serverless:**  
Không tốn công vận hành hạ tầng máy chủ hay cụm cluster.
- **Auto-Scaling tự động:**  
Tự động scale theo tải khi có hàng nghìn request đổ về.
- **Pay-as-you-go:**  
Chỉ trả phí dựa trên số lượng request và chuyển trạng thái thực tế.
- **Tích hợp sâu hệ sinh thái AWS:**  
Kết nối trực tiếp Lambda, S3, DynamoDB, SQS mượt mà.

### ✗ HẠN CHẾ (CONS) & RỦI RO

- **Khó khăn khi Test & Debug Local:**  
Mô phỏng Step Function ở local rất phức tạp, chu kỳ phản hồi phát triển chậm.
- **Chi phí tăng vọt khi Scale tải lớn:**  
Chi phí state transition rất đắt đỏ khi xử lý hàng triệu hóa đơn/tháng.
- **Rủi ro Vendor Lock-in:**  
Bị ràng buộc mã nguồn vào dịch vụ AWS, khó chuyển sang Cloud khác.
- **Tuân thủ On-Premise & Bảo mật:**  
Khách hàng tài chính/ngân hàng yêu cầu dữ liệu nhạy cảm phải xử lý nội bộ.

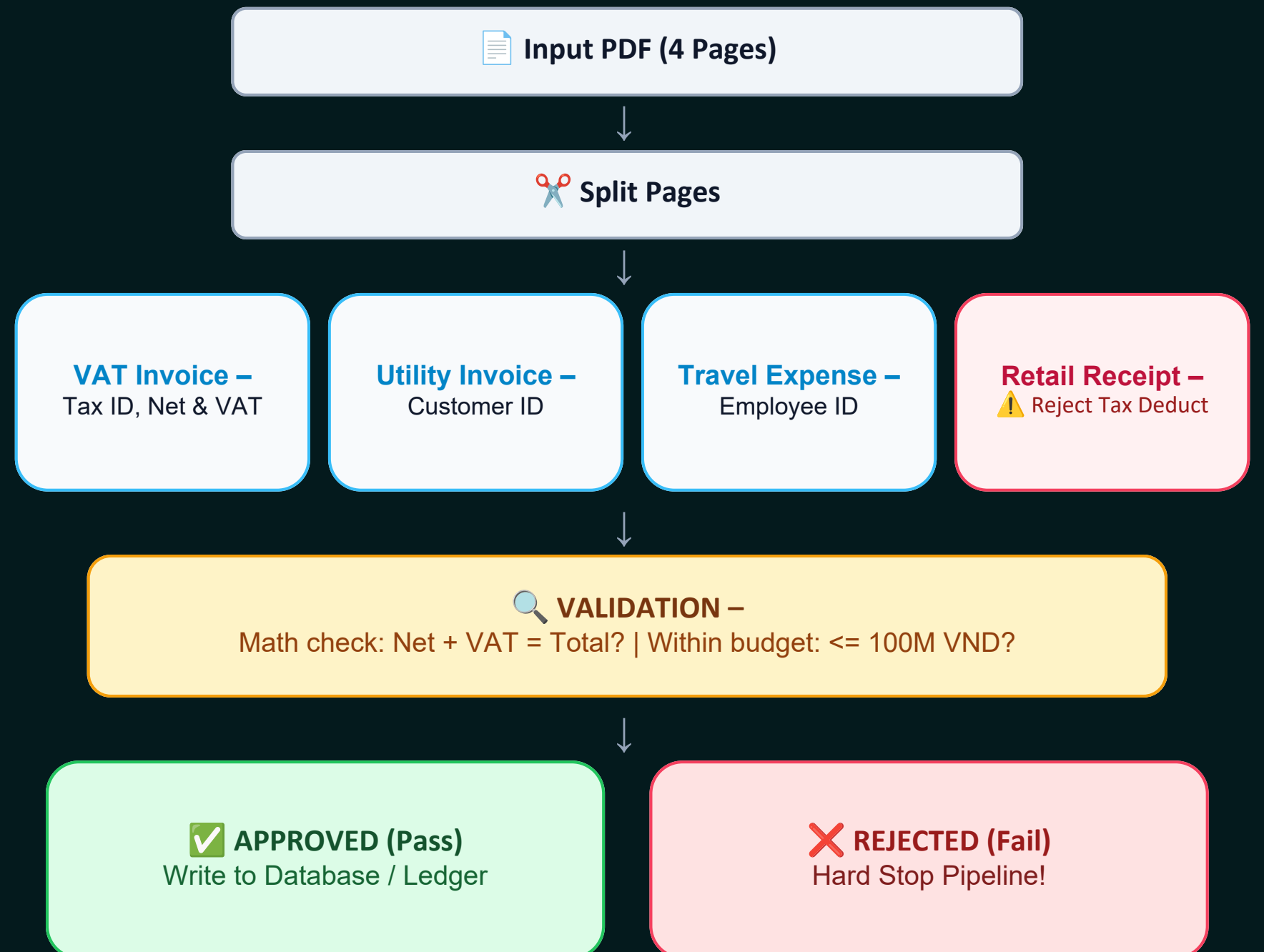


 **Nhiệm vụ** **1. Phân loại 4 trang hóa đơn (4 Invoice Types)**

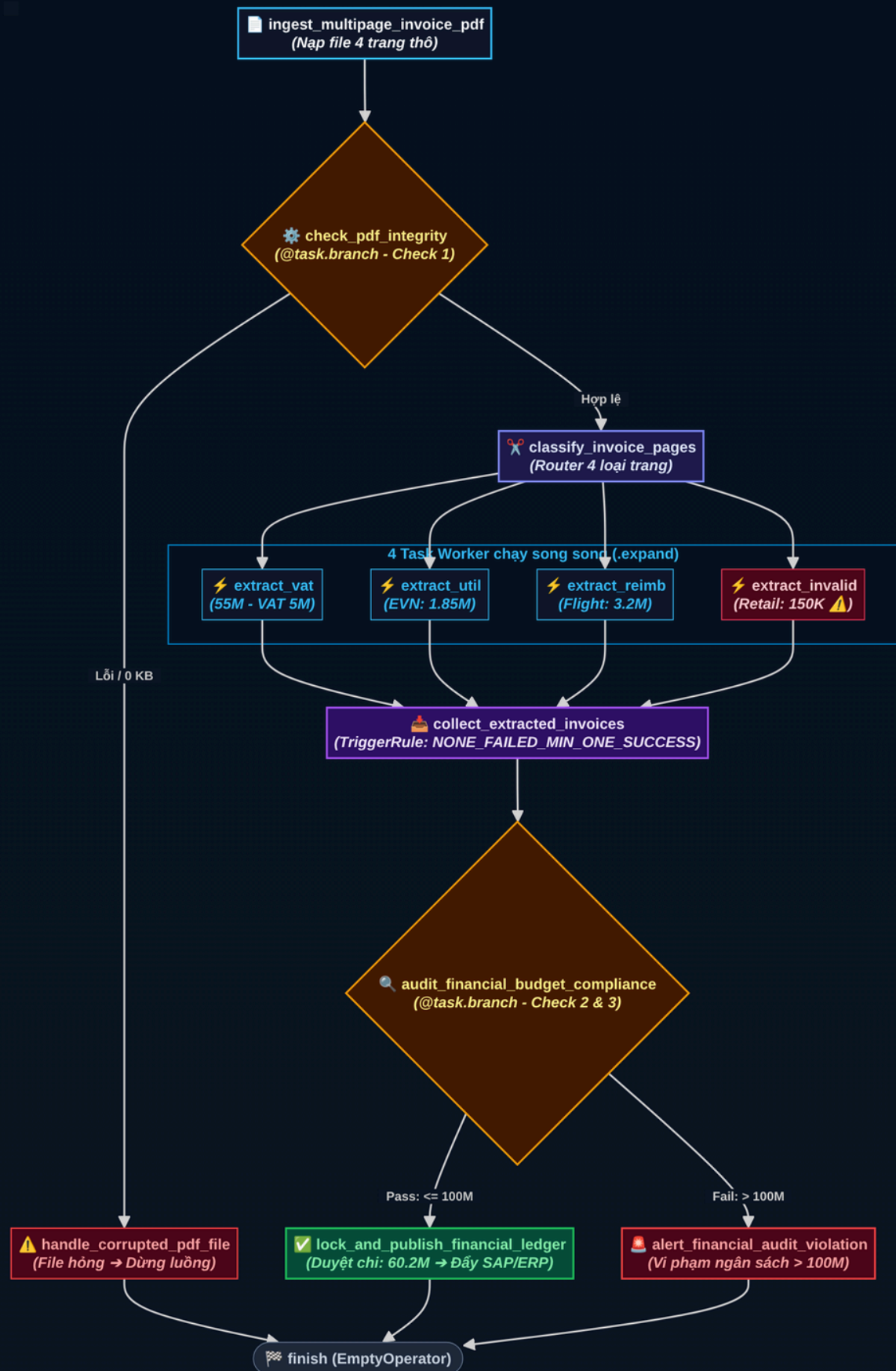
- **VAT Invoice:** Lấy MST, tiền hàng & tiền thuế.
- **Utility Invoice:** Lấy Customer ID để đối chiếu điện/nước.
- **Travel Expense Invoice:** Lấy Employee ID để hoàn tiền công tác.
- **Retail Receipt (No Tax ID):** **Gần cờ cảnh báo** Không duyệt trừ thuế.

 **2. Luồng xử lý & Điều kiện**

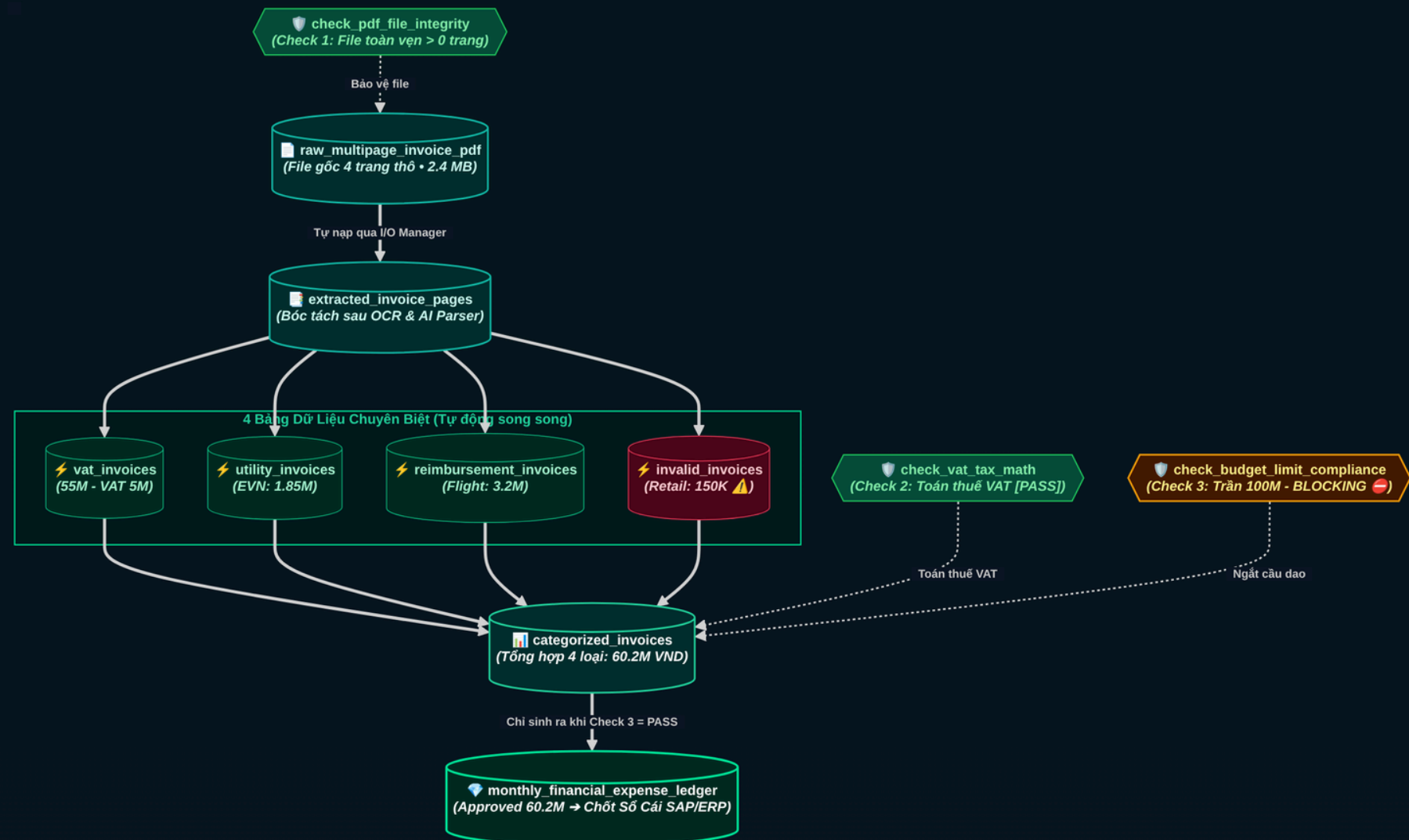
- **VAT Invoice:** Lấy MST, tiền hàng & tiền thuế.
- **Utility Invoice:** Lấy Customer ID để đối chiếu điện/nước.
- **Travel Expense Invoice:** Lấy Employee ID để hoàn tiền công tác.
- **Retail Receipt (No Tax ID):**  **Gần cờ cảnh báo** Không duyệt.

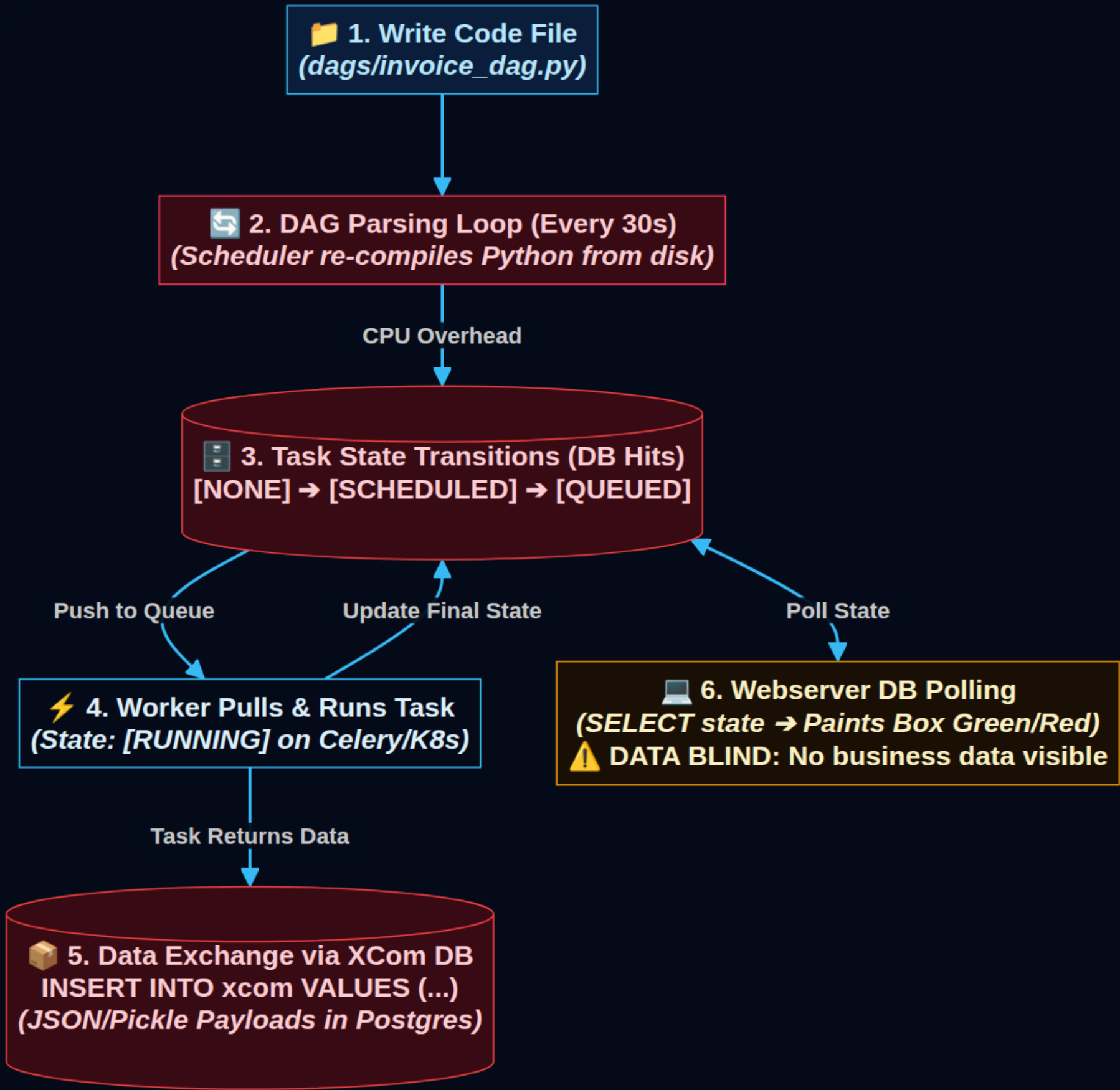


**Bản chất:** Lập trình luồng "How to run" • Rẽ nhánh thủ công bằng @task.branch • Phải set TriggerRule gom nhánh • Dữ liệu ẩn trong XCom DB.



**Bản chất:** Khai báo "What data to produce" • Không task rẽ nhánh, không TriggerRule thủ công • Kiểm định chất lượng là CHIẾC KHIÊN cắm vào Asset • Tự ngắt cầu dao.





🔄 1. Bị Ép Re-Parse Liên Tục (30s/lần)

Scheduler không lưu cấu trúc trên RAM mà liên tục đọc và biên dịch lại code Python. Lỡ import OCR nặng ngoài hàm &rarr; Treo 100% CPU toàn cụm.

🗄️ 2. State Machine Bắn Query DB Dồn Dập

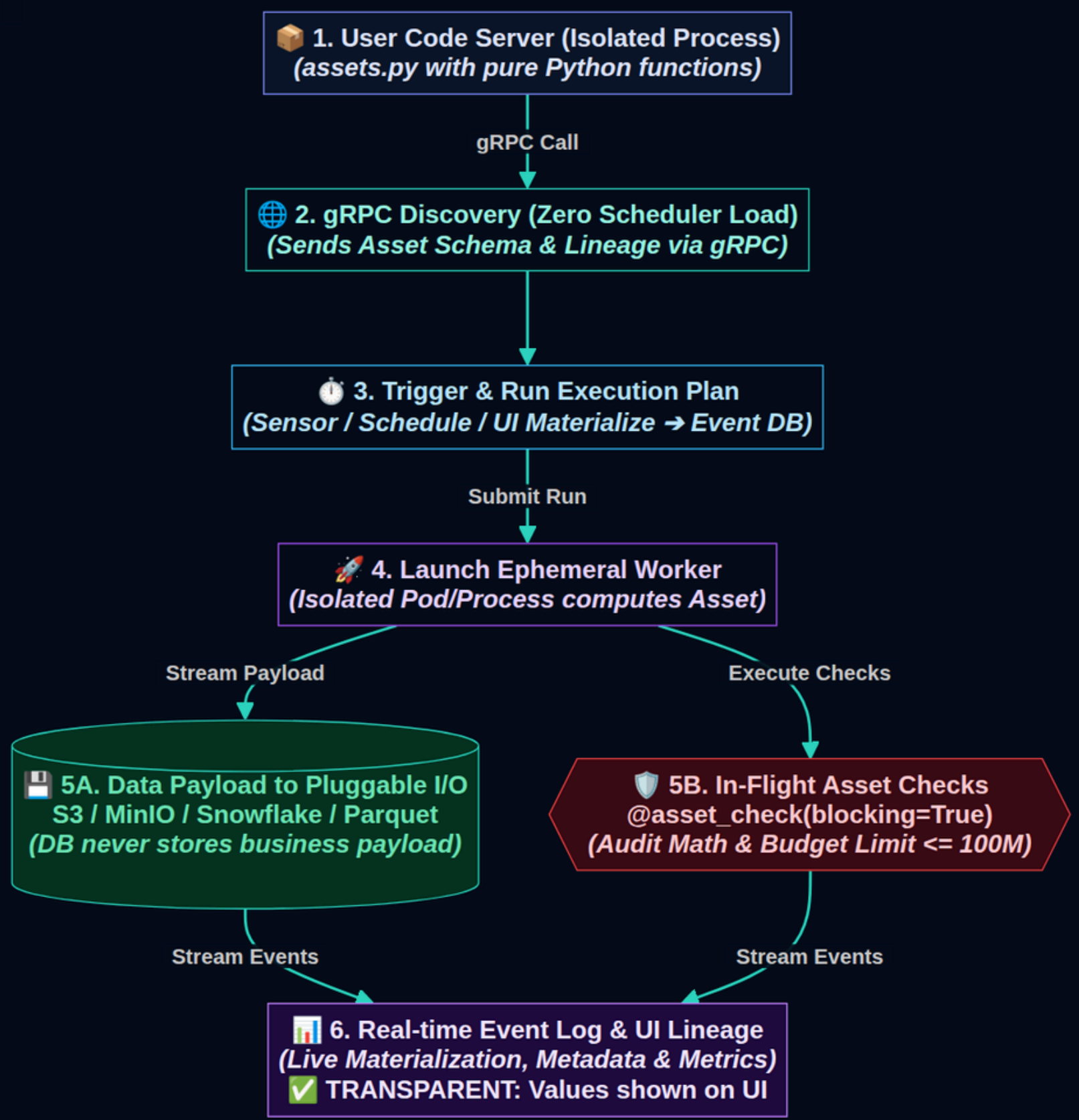
Mỗi bước chuyển trạng thái (None &rarr; Scheduled &rarr; Queued &rarr; Running &rarr; Success) đều UPDATE Postgres &rarr; Nguy cơ lock bảng khi tải lớn.

📦 3. Ép Nạp Dữ Liệu XCom Thẳng Vào Postgres

Khi task return, Airflow serialize rồi INSERT INTO xcom. Dữ liệu lớn (ảnh hóa đơn, DataFrame) làm phình to DB và nghẽn I/O hệ thống.

👁️ 4. Kết Thúc Bằng 'Mù Dữ Liệu' Trên UI

Webserver chỉ đọc DB để tô màu ô vuông Xanh/Đỏ. Kỹ sư hoàn toàn không biết bên trong có bao nhiêu tiền, bao nhiêu hóa đơn nếu không mò log.



 **1. Khám Phá Metadata Qua gRPC (Zero Load)**

Code người dùng chạy trong tiến trình riêng. Nền tảng chỉ dùng gRPC để hỏi schema và lineage, hoàn toàn không nạp code Python của user vào Scheduler.

 **2. Chạy Trong Worker Độc Lập (Ephemeral)**

Mỗi lần chạy bật một container/pod độc lập thực thi hàm Asset thuần khiết. Code có crash hay OOM thì Daemon và Webserver vẫn an toàn 100%.

 **3. I/O Manager Độc Lập & Không Đụng DB**

Dữ liệu trả về được I/O Manager tự động đẩy thẳng lên S3/MinIO hoặc Snowflake. Database hệ thống chỉ lưu nhật ký sự kiện, không lo phình bộ nhớ.

 **4. Asset Checks Đánh Chặn & Báo Cáo Trên UI**

Kiểm định chất lượng dữ liệu được gắn trực tiếp vào Asset. Nếu vi phạm trần 100M sẽ chặn downstream ngay và cảnh báo trên giao diện.

KIẾN TRÚC HẠ TẦNG ON-PREMISE

Ops Overhead

**AIRFLOW (6 THÀNH PHẦN RỜI RẠC) — Cồng Kênh & Dễ Nghềng**

- Stack: Scheduler + Celery Worker Pool + Redis Queue + Postgres HA + Webserver + Triggerer.
- Nỗi đau đĩa chia sẻ: Bắt buộc dựng NFS/EFS chia sẻ để đồng bộ file dags/ giữa các node.
- Xung đột package: Toàn bộ task chung Python env; 1 team upgrade pandas làm gãy pipeline team khác.

**DAGSTER (CHUẨN KUBERNETES & CLOUD-NATIVE) — Tinh Gọn & Độc Lập**

- Stack: Chỉ cần Dagster Webserver + Daemon + PostgreSQL HA (chỉ lưu event metadata nhẹ).
- Lưu trữ đối tượng: Cụm MinIO phân tán làm S3 nội bộ, I/O Manager tự động đẩy/kéo dữ liệu.
- Code Location Pods: Đóng gói Docker riêng, gọi gRPC. Tự do dùng Python 3.10/3.11/3.12 cách ly.

NĂNG LỰC KIỂM THỬ CI/CD & DEVX

Pytest In-Memory

**Airflow: Rào Cản Lớn Khi Viết Unit Test & CI/CD**

- Task gắn chặt với ti, context, dag\_run → Rất khó mock dữ liệu cục bộ.
- Muốn test phải dựng Docker Compose & migrate Postgres (mất 3-5 phút) → Không thể gắn Git Hook.

**Dagster: Unit Test Trong RAM 0.12s (4/4 Tests) — pytest passed**

```
def test_budget_limit_blocks_over_budget():
    res = check_budget_limit_compliance(mock_120m_invoices)
    assert not res.passed # Ngắt luồng bảo vệ trong 0.02s!
```

✨ Tự động kích hoạt kiểm thử trong GitLab CI / GitHub Actions trên từng PR trước khi merge.

KHI NÀO NÊN CHỌN AIRFLOW HAY DAGSTER?

ĐỊNH VỊ NỀN TẢNG

**KHI NÀO NÊN CHỌN APACHE AIRFLOW? [Hạ Tầng Sẵn Có]**

- Điều phối hạ tầng chung (Infra Orchestration):** Bật/tắt máy ảo, trigger job dbt snapshot, gửi email cảnh báo, dọn log hệ thống định kỳ.
- Tác vụ độc lập, thô (Coarse-grained Batch):** Kích hoạt job Spark, Trino, Flink chạy độc lập, không cần trao đổi dữ liệu phức tạp giữa các bước.
- Đã có sẵn cụm On-Premise ổn định:** Doanh nghiệp đã đầu tư VM/Celery với đội ngũ Ops quen vận hành 6 thành phần và ổ đĩa NFS.
- Kiểm thử chấp nhận mô hình truyền thống:** Chấp nhận test gắn chặt với DB (dựng Docker Compose mất 3-5 phút/lần test).

**KHI NÀO NÊN CHỌN DAGSTER? [ĐỀ XUẤT SỐ 1]**

- Thay thế AWS Step Functions On-Premise:** Pipeline phân nhánh nghiệp vụ phức tạp, xử lý hóa đơn, OCR và đối soát theo State Machine.
- Hạ tầng Cloud-Native K8s tinh gọn:** Chỉ 3 service cốt lõi (Webserver, Daemon, Postgres nhẹ). Lưu trữ MinIO S3 nội bộ. Cô lập Pods qua gRPC.
- Kiểm thử tự động CI/CD 0.12s:** Viết unit test in-memory bằng pytest trong RAM, tích hợp mượt mà GitLab CI / GitHub Actions trên mỗi PR.
- Bảo vệ chất lượng dữ liệu:** Tích hợp sẵn Asset Checks (blocking=True) chặn đứng hóa đơn vượt trần 100M ngay trong chu trình chạy.

**Tóm lại:** Phù hợp cho hạ tầng có sẵn, điều phối máy ảo và các luồng xử lý mẽ truyền thống (Cronjob).

**Tóm lại:** Nền tảng toàn diện nhất để thay thế AWS Step Functions trên On-Premise, chuẩn hóa văn hóa Data!